

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Software Engineering Mini-Project

Software Architecture and Design Specification

Online Internship and Placement Preparation Portal

Group: Team 16

Version: 1.0

Date: 05-10-2026

Status: Draft

Team Members

Rahul Bellary: PES1UG24CS357

Dhoni Satwik: PES1UG24CS321

Prateek Hachadad: PES1UG24CS340

Revision History

Version	Date	Author	Change Summary
1.0	06-10-2026	Rahul Bellary	Initial version for Part-1 submission

Approvals

Role	Name	Signature / Date
Team Member	Rahul Bellary	
Team Member	Dhoni Satwik	
Team Member	Prateek Hachadad	
Course Instructor		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Online Internship and Placement Preparation Portal.

1.2 Scope

Covers Student, Recruiter and Admin services: authentication, internship listings, applications, aptitude tests, analytics, notifications and administration.

1.3 Audience

Developers, testers, security reviewers, course instructors and maintainers.

1.4 Definitions

MERN, JWT, RBAC, REST, ADR, TLS, STRIDE. See SRS Appendix A.

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture (components, pattern, technology, risks, traceability, security).
Section 4 describes the detailed design (sequence diagrams, APIs, error handling, UX).

2.2 Related Documents

SRS.docx, Test-Plan.docx, Requirement_List.docx

3. Architecture

3.1 Goals & Constraints

Goals: secure handling of personal data and test integrity, responsive search, 99.5% availability, maintainable modules, support for 5,000 users.

Constraints: MERN stack plus Python analytics, free/open-source tools, one-semester timeline, team of three.

3.2 Stakeholders & Concerns

Stakeholder	Concerns
Students	Usability, fair and reliable tests, privacy of resume and scores
Recruiters	Reaching suitable candidates, easy applicant review
Admin / Placement cell	Moderation, reports, auditability
Developers	Modularity, testability, simple deployment
Instructors	Conformance to the deliverables and quality of design

3.3 Component (UML) Diagram

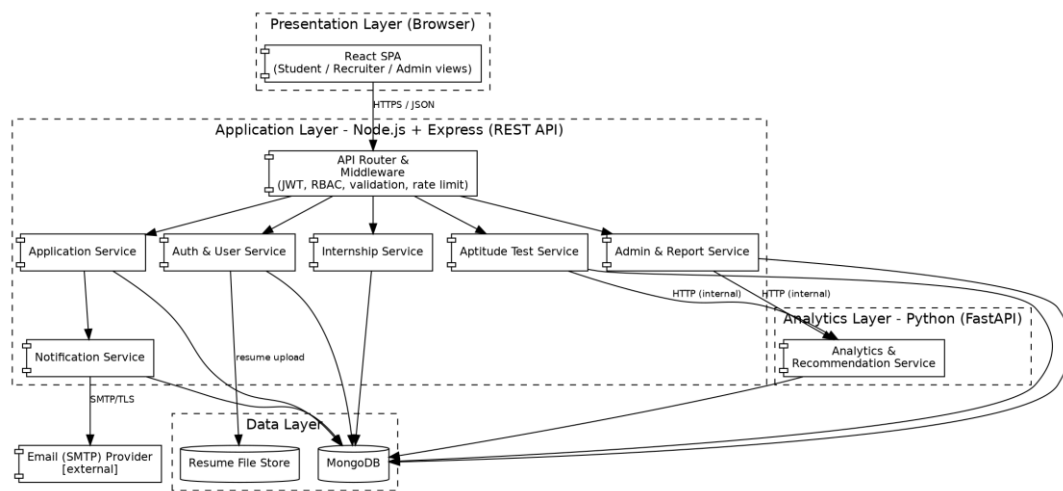


Figure 1: Component diagram of the Portal

3.4 Component Descriptions

Component	Responsibility
React SPA	Role-based UI for Students, Recruiters and Admins; calls REST API over HTTPS.
API Router & Middleware	Single entry point: JWT verification, RBAC, input validation/sanitisation, rate limiting, central error handler.
Auth & User Service	Registration, login, logout, profile and resume upload, user (de)activation.
Internship Service	Listing CRUD, moderation workflow, search and filtering.
Application Service	Apply, duplicate/deadline checks, status updates, applicant review.
Aptitude Test Service	Question bank, attempt lifecycle, timer, autosave, grading, review.
Notification Service	In-app notifications and email via SMTP; asynchronous dispatch.
Admin & Report Service	Audit log, CSV export, aggregate queries.
Analytics & Recommendation Service (Python)	Computes score trends, weak categories, recommendations and aggregates.

MongoDB / Resume store	Persistent data and uploaded PDF resumes.
------------------------	---

3.5 Chosen Architecture Pattern and Rationale

Pattern: layered architecture (presentation, application, data) with a modular monolith for the Node.js backend and one separate Python analytics service. Rationale: clear separation of concerns, simple deployment for a three-member team, and independent evolution of the analytics logic in Python where data libraries are strong. Full microservices were rejected as over-complex for the expected scale (5,000 users); a single monolith without the analytics split was rejected because the analytics requirement is naturally implemented in Python (ADR-1, ADR-2).

ADR	Decision	Alternatives considered
ADR-1	Layered modular monolith for core API	Microservices (operational overhead too high)
ADR-2	Separate Python FastAPI analytics service	Analytics in Node.js (weaker data tooling)
ADR-3	MongoDB document store	Relational DB (schema flexibility for questions/attempts preferred)
ADR-4	Stateless JWT sessions	Server-side sessions (harder to scale horizontally)

3.6 Technology Stack & Data Stores

Layer	Technology
Client	React, React Router, Chart.js
API	Node.js 20, Express, Mongoose, JWT, bcrypt, helmet, express-rate-limit, express-validator
Analytics	Python 3.11, FastAPI, pandas
Data	MongoDB 7; resume files on a protected storage volume
Transport	TLS 1.2+ (HTTPS), SMTP over TLS
DevOps	Docker, GitHub Actions, Jest, pytest, Cypress

3.7 Risks & Mitigations

Risk	Mitigation
Analytics service failure	Core flows (apply, test grading) do not depend on it; dashboard shows cached values and a notice
Database failure or data loss	Daily backups; restore procedure tested
Test answer leakage	Answer key never sent during an active attempt; grading only on the server
SMTP failure	Queued retry; in-app notification remains
Performance degradation under load	Database indexes, pagination, stateless API for horizontal scaling

3.8 Traceability to Requirements

Requirement	Component
PIP-F-001 to PIP-F-003, PIP-F-021	Auth & User Service
PIP-F-004 to PIP-F-007	Internship Service
PIP-F-008 to PIP-F-010	Application Service
PIP-F-011	Notification Service
PIP-F-012 to PIP-F-016	Aptitude Test Service
PIP-F-017 to PIP-F-019	Analytics Service, Admin & Report Service
PIP-F-020, PIP-F-022	Admin & Report Service
PIP-NF-001, 002, 006	Layered stateless API, indexes, pagination

PIP-NF-003, 008	Deployment, backup (SAD 3.7)
PIP-NF-004, 005	React SPA (UX, SAD 4.5)
PIP-S-001 to PIP-S-007	API Router & Middleware, Security Architecture (SAD 3.9)

3.9 Security Architecture

Threat modelling (STRIDE):

Threat	Example	Mitigation	Requirement
Spoofing	Stolen or guessed credentials	bcrypt, lockout after 5 failures, short-lived JWT	PIP-F-002, PIP-S-002
Tampering	Modifying application status or scores through the API	RBAC, server-side validation, grading only on server	PIP-S-003, PIP-S-004
Repudiation	Denying an approval or status change	Append-only audit log	PIP-S-007, PIP-F-022
Information disclosure	Resume or answer key leakage	TLS, ownership checks, answers withheld during attempt	PIP-S-001, PIP-S-006, PIP-F-015
Denial of service	Request flooding	Rate limiting (HTTP 429), pagination	PIP-S-005
Elevation of privilege	Student calling Admin endpoints	RBAC middleware on every route	PIP-S-003

4. Design

4.1 Design Overview

Each backend module follows route -> controller -> service -> model layering. Middleware handles authentication, authorisation, validation and error formatting. The React client uses role-based routes and a shared API client.

4.2 UML Sequence Diagrams

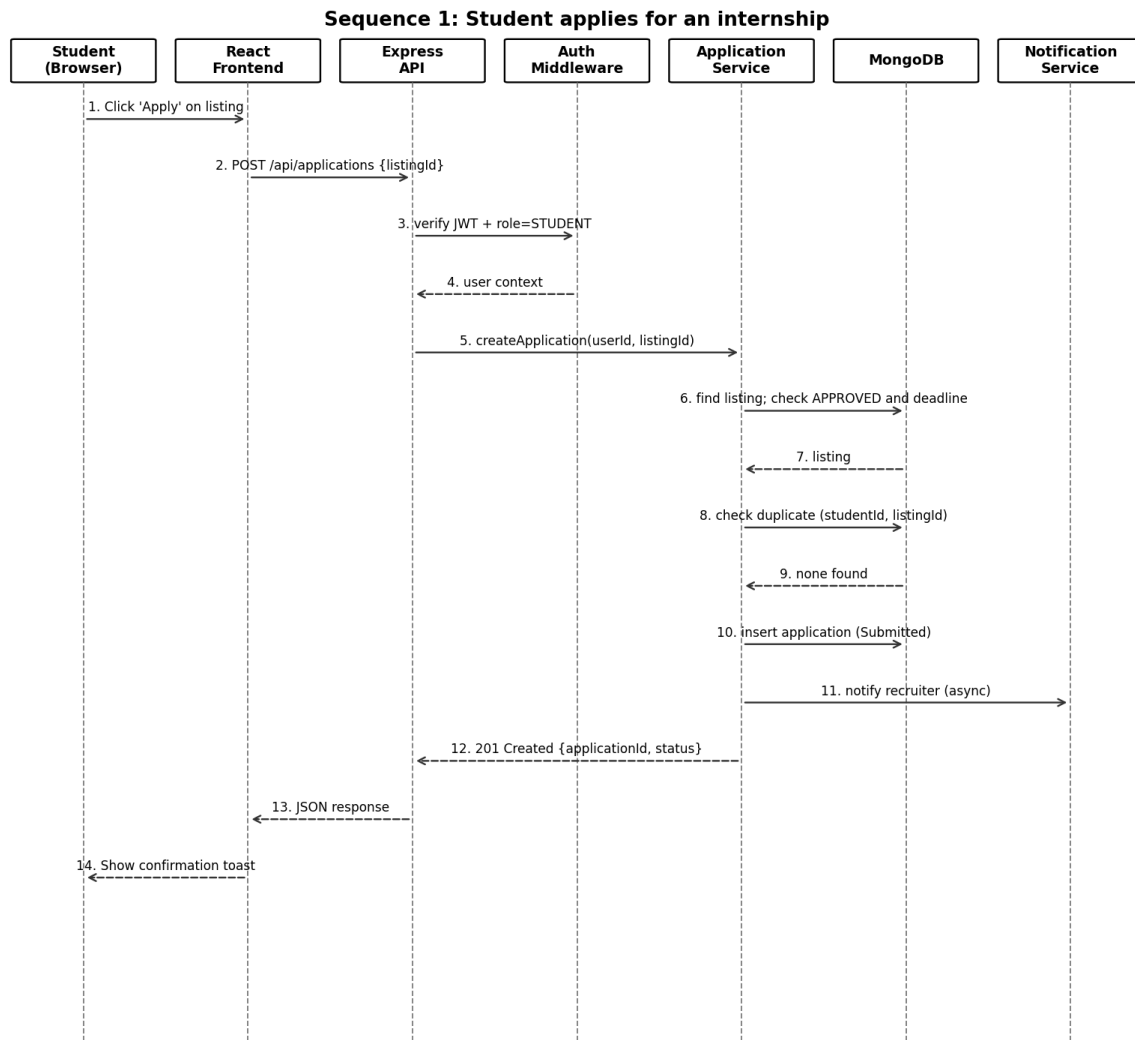


Figure 2: Sequence diagram - student applies for an internship

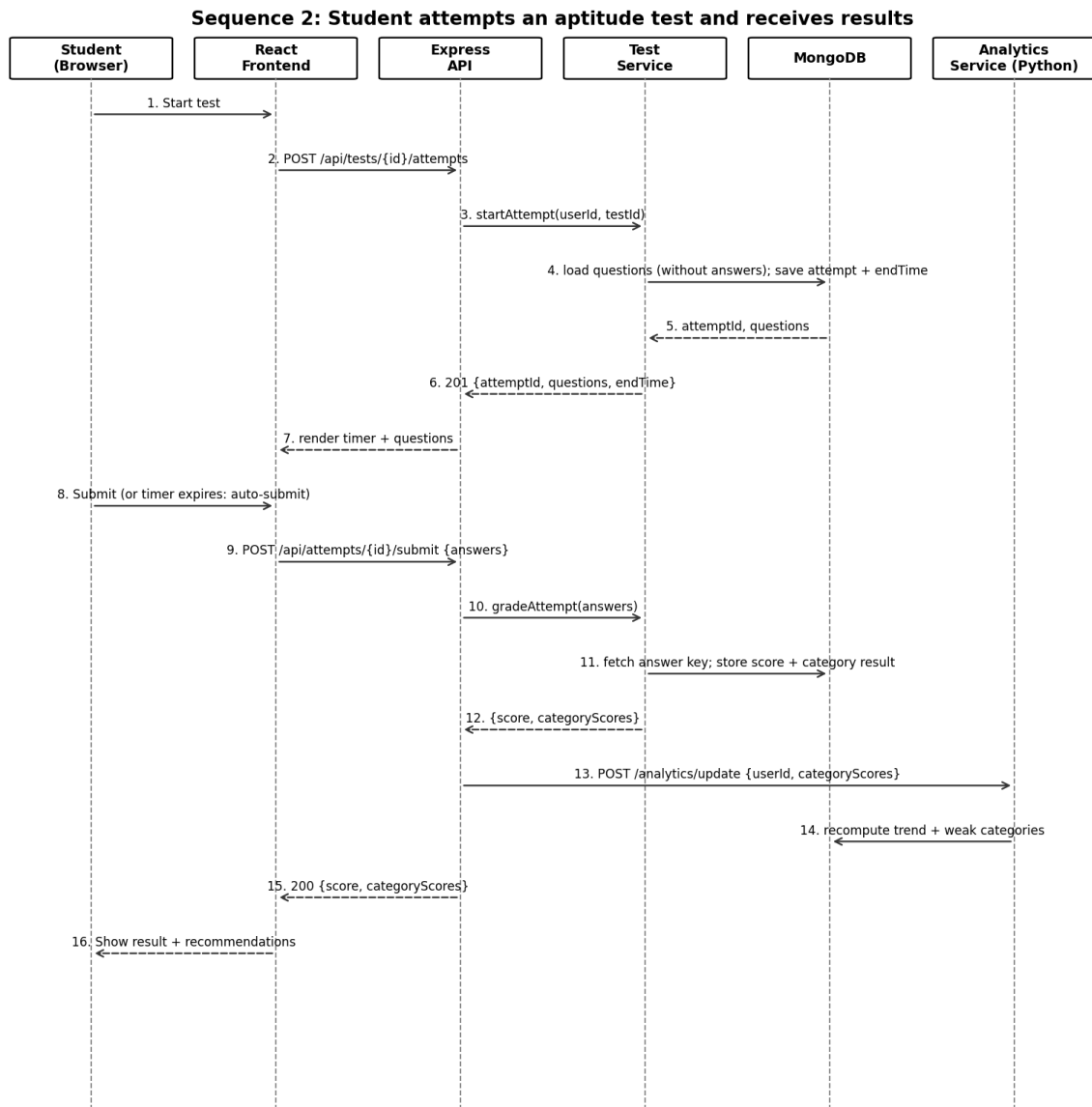


Figure 3: Sequence diagram - aptitude test attempt and results

4.3 API Design

All endpoints use JSON over HTTPS and (except register/login) require the header Authorization: Bearer <JWT>.

Auth API

Endpoint	Method	Request	Response	Errors
/api/auth/register	POST	{name, email, password, role}	201 {userId}	400 invalid input, 409 email exists
/api/auth/login	POST	{email, password}	200 {token, role, expiresIn}	401 invalid credentials, 423 account locked

Internship API

Endpoint	Method	Request	Response	Errors
----------	--------	---------	----------	--------

/api/internships	GET	query: q, skill, location, mode, minStipend, maxStipend, page	200 {items[], page, total}	400 invalid filter
/api/internships	POST (Recruiter)	{title, company, description, skills, stipend, location, mode, deadline}	201 {id, status: Pending}	400, 403
/api/internships/{id}/review	PATCH (Admin)	{decision, remark}	200 {id, status}	403, 404

Application API

Endpoint	Method	Request	Response	Errors
/api/applications	POST (Student)	{listingId}	201 {applicationId, status: Submitted}	400 no resume, 403, 404, 409 duplicate, 410 deadline passed
/api/applications/mine	GET (Student)	-	200 {items[]}	401
/api/applications/{id}/status	PATCH (Recruiter)	{status}	200 {id, status}	403, 404, 422 invalid transition

Test API

Endpoint	Method	Request	Response	Errors
/api/tests/{id}/attempts	POST (Student)	-	201 {attemptId, questions[], endTime}	403, 404, 409 active attempt exists
/api/attempts/{id}/answers	PUT	{answers[]}	204	403, 410 attempt closed
/api/attempts/{id}/submit	POST	{answers[]}	200 {score, categoryScores}	403, 404, 409 already submitted

Analytics API (internal, Python)

Endpoint	Method	Request	Response	Errors
/analytics/update	POST	{userId, categoryScores}	202 accepted	422 invalid body
/analytics/students/{id}/summary	GET	-	200 {trend[], categoryScores, recommendations[]}	404

4.4 Error Handling, Logging & Monitoring

- Standard error body: {error: {code, message, details?}} with correct HTTP status (400, 401, 403, 404, 409, 410, 422, 423, 429, 500).
- Central Express error handler; no stack traces or internal details returned to clients.
- Logs (winston/pino) in structured JSON with request ID; passwords, tokens and resume content are never logged.
- Audit log for security-relevant events (PIP-S-007, PIP-F-022).
- Monitoring: API latency percentiles, error rate, analytics service health check, uptime checks and alerts.

4.5 UX Design

- Responsive layout from 360 px; accessible contrast and labelled form fields.
- Role-based dashboards; search with clear filters; application status shown as coloured badges with text.

- Test screen: countdown timer, question navigator, autosave indicator, confirmation before submit.

4.6 Open Issues & Next Steps

- Choice of file storage (local volume vs object storage) to be finalised.
- Future enhancements: resume parsing and scoring, interview scheduling,
- Creating the application further on.

5. Appendices

5.1 Glossary: MERN, JWT, RBAC, REST, ADR, TLS, STRIDE (see SRS Appendix A).

5.2 References: IEEE 42010, OWASP ASVS, NIST SP 800-160.

5.3 Tools: draw.io / PlantUML for diagrams, Postman / Swagger for API documentation.